

Step 1: Install Required Libraries

What are we performing?

- Installing the required LangChain packages.

Why are we performing it?

- These libraries are required to build the RAG pipeline.

Difference from Previous Notebook

- This notebook uses LangChain's FAISS implementation instead of manually creating a FAISS index.

```
In [3]: # pip install langchain_huggingface
# !pip install langchain_community
```

Step 2: Import Required Libraries

What are we performing?

- Importing libraries for text preprocessing, chunking, embeddings and vector database.

Why are we performing it?

- These libraries are used throughout the RAG pipeline.

Difference from Previous Notebook

- `HuggingFaceEmbeddings` and `FAISS` from LangChain are used instead of manual embedding generation and FAISS indexing.

```
In [4]: from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
import spacy
import faiss
```

```
c:\Users\LENOVO\AppData\Local\Programs\Python\Python314\Lib\site-packages\langchain_core\utils\pydantic.py:41: UserWarning: Core Pydantic V1 functionality isn't compatible with Python
3.14 or greater.
  from pydantic.v1 import BaseModel as BaseModelV1
c:\Users\LENOVO\AppData\Local\Programs\Python\Python314\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
C:\Users\LENOVO\AppData\Local\Temp\ipykernel_19440\535571621.py:3: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See
https://github.com/langchain-ai/langchain-community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.vectorstores import FAISS
```

Step 3: Load the Document

What are we performing?

- Reading the text file.
- Storing it in the variable `data`.

Why are we performing it?

- This document acts as the knowledge source for the RAG system.

```
In [5]: data = open(r'machine_learning_2000_sentences.txt').read()
```

\$\$ Text Normalization \$\$

What are we performing?

- Preparing the text for preprocessing.

Why are we performing it?

- Clean text improves chunk quality and embedding quality.

Step 4: Convert Text to Lowercase

What are we performing?

- Converting all characters to lowercase.

Why are we performing it?

- Creates a consistent text format.

In [6]:

```
data = data.lower()
```

Step 5: Remove Extra Spaces

What are we performing?

- Removing multiple spaces using Regex.

Why are we performing it?

- Produces cleaner text.

In [7]:

```
import re
data = re.sub(r'\s{2,}',' ', data)
```

Step 6: Remove Statement Labels

What are we performing?

- Removing patterns like Machine Learning Statement 1: .

Why are we performing it?

- These labels do not contribute to the meaning of the document.

In [8]:

```
data = re.sub(r'machine learning statement \d+:','',data)
data = re.sub(r'machine learning\nstatement \d+:','',data)
data = re.sub(r'machine\nlearning statement \d+:','',data)
```

Step 7: Expand Contractions

What are we performing?

- Importing the contractions library.
- Replacing contractions with their full form.

Why are we performing it?

- Used to expand words such as can't → cannot .
- Improves text consistency.

```
In [9]: import contractions
data = contractions.fix(data)
```

Step 8: Remove Punctuation

What are we performing?

- Importing punctuation symbols.
- Removing punctuation using Regex.

Why are we performing it?

- Keeps only meaningful words.

```
In [10]: data = re.sub(r'^\d-9a-zA-Z\s','',data)
```

Step 12: Spell Correction

What are we performing?

- Importing TextBlob.

Why are we performing it?

- Used for correcting spelling mistakes if required.

```
In [11]: # from textblob import TextBlob
# str(TextBlob(data).correct())
```

Step 13: Lemmatization using spaCy

What are we performing?

- Importing spaCy.

Why are we performing it?

- Used for tokenization, stop-word removal and lemmatization.

```
In [12]: nlp = spacy.load('en_core_web_sm')
```

Step 14: Tokenize and Lemmatize

What are we performing?

- Splitting text into tokens.
- Removing stop words.
- Converting words to their root form.

Why are we performing it?

- Produces cleaner text for embedding generation.

```
In [13]: tokens = nlp(data)
updated_tokens = [token.lemma_ for token in tokens if not token.is_stop]
```


Step 15: Convert Tokens Back to Text

What are we performing?

- Joining processed tokens into a single string.

Why are we performing it?

- Creates the final cleaned document.

```
In [14]: data = ' '.join(updated_tokens).strip()
```

Step 16: Chunk the Document

What are we performing?

- Creating a text splitter.

Why are we performing it?

- Splits the document into smaller chunks for retrieval.

```
In [15]: splitter = RecursiveCharacterTextSplitter(
         chunk_size = 100, chunk_overlap = 20
         )
```

Step 14: Create Document Chunks

What are we performing?

- Creating LangChain `Document` objects.

Why are we performing it?

- LangChain stores both content and metadata.

Difference from Previous Notebook

- Previous notebook used `split_text()`.
- This notebook uses `create_documents()`, which returns `Document` objects.

```
In [16]: chunks = splitter.create_documents([data])
```

Step 15: Display the First Chunk

What are we performing?

- Printing the first chunk.

Why are we performing it?

- To verify chunk creation.

```
In [17]: chunks[0]
```

Out[17]: Document(metadata={}, page_content='workflow emphasizing unsupervise \n learning improve carefully validate feature scale')

Step 16: Check the Chunk Type

What are we performing?

- Checking the datatype of a chunk.

Why are we performing it?

- Confirms that each chunk is a LangChain `Document`.

```
In [18]: type(chunks[0])

Out[18]: langchain_core.documents.base.Document
```

Step 17: Display Chunk Content

What are we performing?

- Printing only the text inside the chunk.

Why are we performing it?

- To inspect the actual chunk content.

```
In [19]: print(chunks[0].page_content)

        workflow emphasizing unsupervise
        learning improve carefully validate feature scale
```

Step 18: Add Metadata to Chunks

What are we performing?

- Adding metadata to each document chunk.

Why are we performing it?

- Helps identify the source of the retrieved chunk.
- Useful when retrieving data from multiple documents.

```
In [20]: chunks[0].metadata = 'machine_learning_2000_sentences.txt'

In [21]: chunks[0]

Out[21]: Document(metadata='machine_learning_2000_sentences.txt', page_content='workflow emphasizing unsupervise \n learning improve carefully validate feature scale')
```

```
In [22]: chunks[0].metadata = {'file_name': 'machine_learning_2000_sentences.txt'}

In [23]: chunks[0]

Out[23]: Document(metadata={'file_name': 'machine_learning_2000_sentences.txt'}, page_content='workflow emphasizing unsupervise \n learning improve carefully validate feature scale')
```

Step 19: Load the Embedding Model

What are we performing?

- Loading the Hugging Face embedding model.

Why are we performing it?

- Converts document chunks into vector embeddings for semantic search.

Difference from Previous Notebook

Previous notebook:
Used SentenceTransformer().encode()

This notebook:
Uses LangChain's HuggingFaceEmbeddings.

In [24]:

embedding_model = HuggingFaceEmbeddings(
 model_name="sentence-transformers/all-mpnet-base-v2"
)

Loading weights: 100%|██████████| 199/199 [00:00<00:00, 2259.37it/s]

Step 20: Create the LangChain FAISS Vector Store

What are we performing?

- Creating a FAISS vector database from the document chunks.

Why are we performing it?

- Stores embeddings efficiently.
- Enables semantic similarity search.

Difference from Previous Notebook

Previous notebook:
Manually generated embeddings.
Manually created the FAISS index.

> Syntax:

python

chunk_embeddings = embedding_model.encode(chunks).astype('float32')
dimension = chunk_embeddings.shape[1]
faiss.normalize_L2(chunk_embeddings)
index_faiss_db = faiss.IndexFlatIP(dimension)
index_faiss_db.add(chunk_embeddings)

This notebook:
LangChain performs both operations automatically.

In [25]:

vector_db = FAISS.from_documents(
 documents=chunks,
 embedding=embedding_model
)

In [26]:

vector_db

Out[26]: <langchain_community.vectorstores.faiss.FAISS at 0x2d17f5e2ba0>

Step 21: Perform Similarity Search

What are we performing?

- Searching the vector database using the user's query.

Why are we performing it?

- Retrieves the most relevant chunks based on semantic similarity.

Difference from Previous Notebook

Previous notebook: Used `index_faiss_db.search()` .

This notebook: Uses `vector_db.similarity_search()` .

```
In [27]: user_query = "What is Machine Learning?"
        r_chunks = vector_db.similarity_search(user_query)

In [28]: r_chunks

Out[28]: [Document(id='b99692a4-166b-4416-ab08-3a48b09b6af6', metadata={}, page_content='supervise learning improve carefully validate supervised'),
          Document(id='17fc7acc-32e5-49f9-8cf2-be7a6618987f', metadata={}, page_content='supervise learning improve carefully validate supervised'),
          Document(id='9c0b2727-5e33-47d4-ae6-b18720ed52ec', metadata={}, page_content='supervise learning improve carefully validate supervised'),
          Document(id='9ab85c37-e869-4d57-865f-b4b9fdb0eb24', metadata={}, page_content='supervise learning improve carefully validate supervised')]
```

Step 22: Display Retrieved Chunks

What are we performing?

- Printing every retrieved chunk.

Why are we performing it?

- To verify that the retrieved information is relevant.

```
In [29]: for chunk in r_chunks:
        print(chunk.page_content)

        supervise learning improve carefully validate supervised
        supervise learning improve carefully validate supervised
        supervise learning improve carefully validate supervised
        supervise learning improve carefully validate supervised
```

Step 23: Merge Retrieved Chunks

What are we performing?

- Combining all retrieved chunks into one text.

Why are we performing it?

- Creates the context that will be sent to the LLM.

```
In [30]: updated_r_chunks = set()
        for chunk in r_chunks:
            updated_r_chunks.add(chunk.page_content)

        R_Text = '\n'.join(updated_r_chunks)

In [31]: R_Text

Out[31]: 'supervise learning improve carefully validate supervised'

In [33]: context = "\n".join(
        list(set(chunk.page_content for chunk in r_chunks))
        )
```

Step 23: Generate the Final Response using Hugging Face API

What are we performing?

- Creating a function to generate the final answer using the retrieved context.
- Retrieving the most relevant chunks from the LangChain FAISS vector database.
- Creating a prompt using the retrieved context and the user's query.
- Sending the prompt to a Hugging Face hosted Large Language Model (LLM).
- Returning the generated response.

Why are we performing it?

- Retrieval provides only the relevant context, not the final answer.
- The LLM reads the retrieved context and generates a human-readable response.
- This completes the **Generation** stage of the Retrieval-Augmented Generation (RAG) pipeline.

> Difference from Previous Notebook

- >
- > - Previous notebook used a separate retrieval function (`r_search()`) followed by a generation function (`g_text()`).
- > - In this notebook, LangChain's `similarity_search()` performs retrieval directly inside the generation function, reducing the amount of code.


```
In [39]: # Import the os module to access environment variables and the requests library to send HTTP requests
import os
import requests

# Hugging Face Router API endpoint
API_URL = "https://router.huggingface.co/v1/chat/completions"

# Create the request headers containing the API token
headers = {
    "Authorization": f"Bearer {os.environ['HF_TOKEN']}",
}

# Function to perform Retrieval + Generation
def generate_response(user_query):
    # ----- Retrieval -----

    # Retrieve the most relevant document chunks from the FAISS vector database
    r_chunks = vector_db.similarity_search(user_query)

    # Extract only the text (page_content) from each retrieved Document object
    # Remove duplicate chunks using set()
    # Join all chunks into a single context string
    context = "\n".join(
        list(set(chunk.page_content for chunk in r_chunks))
    )

    # ----- Prompt -----
    # Create the prompt that will be sent to the language model
    prompt = f"""
You are a helpful AI assistant.
Use context to answer:
{context}

Output Structure:
Input: {user_query}
Output: structured output
"""

    # Create the JSON payload for the API request
    payload = {
        "model": "deepseek-ai/DeepSeek-V3:novita",
        "messages": [
            {
                "role": "user",
                "content": prompt
            }
        ]
    }

    # # USE THESE MODELS
    # # DeepSeek-V3
    # # Qwen-Instruct
    # # Llama Instruct
    # # Mistral Instruct

    # Send the POST request to the Hugging Face API
    response = requests.post(
        API_URL,
        headers=headers,
        json=payload,
        timeout=90
    )

    # Convert the JSON response into a Python dictionary
    result = response.json()

    # Return only the generated answer from the response
    return result["choices"][0]["message"]["content"]

# User's question
user_query = "Explain Machine Learning"

# Generate the response using the RAG Pipeline
answer = generate_response(user_query)

# Display the final answer
print(answer)
```

Here’s a structured output based on your input context:

```
### **Structured Output: Explain Machine Learning**

#### **1. Unsupervised Learning**
- **Workflow Emphasis**:
  - Focus on discovering hidden patterns or structures in unlabeled data.
- **Improvement & Validation**:
  - **Improve**: Feature scaling (e.g., normalization, standardization) to ensure algorithms (e.g., k-means, PCA) perform optimally.
  - **Validate**: Use metrics like silhouette score, inertia, or domain-specific evaluation to assess cluster quality.

#### **2. Supervised Learning**
- **Workflow Emphasis**:
  - Leverages labeled data to train models for prediction/classification.
- **Improvement & Validation**:
  - **Improve**: Feature engineering, hyperparameter tuning, and addressing overfitting (e.g., regularization).
  - **Validate**: Metrics like accuracy, precision, recall, F1-score, or AUC-ROC; cross-validation for robustness.

#### **Key Cross-Cutting Themes**
- **Feature Scaling**: Critical for both types (e.g., SVM, neural networks in supervised; k-means in unsupervised).
- **Validation**: Rigorous testing ensures generalizability (e.g., train-test splits, bootstrapping).

#### **Example Workflow**
1. **Unsupervised**: Scale features → Cluster (k-means) → Validate with silhouette analysis.
2. **Supervised**: Scale features → Train (Random Forest) → Validate via 5-fold CV.

Let me know if you'd like to expand on any section!
```

In []: